

# Reviewer Pack — Minimal Rank of Grothendieck-Tensor Product over XOR-AND Tre...

Entry ffb7ed742253 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 17:30:40 UTC

## Minimal Rank of Grothendieck-Tensor Product over XOR-AND Tree Width

Entry ID: ffb7ed742253 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 17:30:40 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebraic Geometry (Grothendieck Tensor Product) **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

#### Statement:

[‘For a given XOR-AND tree  $T$  with width  $w$  and depth  $d$ , the minimal rank of the Grothendieck tensor product of its clause-indicator polynomials modulo 2 is  $O(w^d)$ .’, ‘Equivalently, for any polynomial  $f: \{0, \dots, n-1\} \rightarrow \mathbb{Z}_2$  such that  $f[0] = 1$  and  $f[i] = 0$  if  $i$  corresponds to a satisfying assignment in  $T$ , the rank of the Grothendieck tensor product of the ideals generated by  $f$  and its negations is  $O(w^d)$ .’, ‘No polynomial  $g: \{0, \dots, n-1\} \rightarrow \mathbb{Z}_2$  exists with  $g[0] = 1$  and  $g[i] \neq f[i]$  for all  $i$  such that  $\text{rank}(g) < w^d$ .’]

#### Rationale (proposer’s reasoning):

[‘The Grothendieck tensor product provides a natural way to capture the complexity of algebraic objects, which may expose the inherent structure in XOR-AND trees.’, ‘If the conjecture holds, it would suggest a deep connection between the algebraic geometry and complexity theory, potentially leading to new algorithms or lower bounds.’, ‘The minimal rank of Grothendieck tensor products has not been extensively studied in the context of complexity theory, making this a novel approach.’]

**Taxonomy category:** TROPICAL\_FOURIER\_ANALYSIS (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 3ba0a3638897d0bb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if for all generated XOR-AND trees with width  $w$  and depth  $d$ , the ratio of the rank  $r$  of the Grothendieck tensor product over  $w^d$  is  $\leq 1.5$  with a confidence interval of 95% using at least 50 seeds.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 1.00       | UNCERTAIN        | HITS             |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | UNCERTAIN | 1.00       | HITS             | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - 'Grothendieck tensor product' AND 'XOR-AND tree width' AND minimal rank' - 'clause-indicator polynomials' AND 'Grothendieck tensor product' mod 2 AND 'tree width' - 'ideals generated by polynomial' AND 'negations' AND 'rank 0(w^d) XOR-AND

**Top relevant hits considered:** - [http://arxiv.org/abs/1607.03608v3] On the tensor product of linear sites and Grothendieck categories - [http://arxiv.org/abs/2410.05768v1] My view on and experience with Grothendieck's anabelian geometry - [http://arxiv.org/abs/2107.02219v2] Grothendieck Rings of Queer Lie Superalgebras - [http://arxiv.org/abs/1502.02753v4] Ideal Tree-drawings of Approximately Optimal Width (And Small Height) - [http://arxiv.org/abs/1805.01816v2] Polynomials and tensors of bounded strength - [http://arxiv.org/abs/2412.14390v2] Using SimTeX to simplify polynomial expressions with tensors - [http://arxiv.org/abs/1807.08021v2] On ideals generated by fold products of linear forms - [http://arxiv.org/abs/1701.03503v3] Monomial generators of complete planar ideals

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def xor_and_tree(depth, width):
        if depth == 0:
            return [random.choice([0, 1])]
        else:
            left = xor_and_tree(depth - 1, width // 2)
            right = xor_and_tree(depth - 1, width // 2)
            return [l ^ r for l in left for r in right]

    def polynomial(f):
        n = len(f)
        poly = [[0] * (n + 1) for _ in range(n + 1)]
        poly[0][0] = 1
        for i in range(1, n + 1):
            poly[i][0] = -f[i - 1]
```

```

        for j in range(1, i + 1):
            poly[i][j] = poly[i - 1][j - 1] ^ poly[i - 1][j]
    return poly

def grothendieck_tensor_product(poly1, poly2):
    n = len(poly1)
    m = len(poly2)
    result = [[0] * (n + m) for _ in range(n + m)]
    for i in range(n):
        for j in range(m):
            for k in range(n + m):
                result[i + j][k] ^= poly1[i][j] & poly2[k - j]
    return result

def rank(matrix):
    n = len(matrix)
    m = len(matrix[0])
    row, col = 0, 0
    while row < n and col < m:
        if matrix[row][col] == 0:
            for i in range(row + 1, n):
                if matrix[i][col] != 0:
                    matrix[row], matrix[i] = matrix[i], matrix[row]
                    break
            else:
                col += 1
                continue
        pivot = matrix[row][col]
        for j in range(col, m):
            matrix[row][j] ^= pivot
        for i in range(n):
            if i != row and matrix[i][col] != 0:
                factor = matrix[i][col]
                for j in range(col, m):
                    matrix[i][j] ^= factor & matrix[row][j]
        row += 1
        col += 1
    return min(row, col)

def clause_indicator_polynomial(tree):
    n = len(tree)
    poly = [0] * (n + 1)
    for i in range(n):
        if tree[i] == 1:
            poly[0] ^= 1
            for j in range(1, n + 1):
                poly[j] ^= poly[j - 1]
    return poly

def negate_polynomial(poly):
    return [1 ^ x for x in poly]

depth = random.randint(2, 4)
width = 2 ** depth
tree = xor_and_tree(depth, width)
f = clause_indicator_polynomial(tree)

```

```

g = negate_polynomial(f)

poly_f = polynomial(f)
poly_g = polynomial(g)
tensor_product = grothendieck_tensor_product(poly_f, poly_g)

rank_value = rank(tensor_product)
w_d = width ** depth

return {
    "metric_name": "Ratio of Rank to w^d",
    "metric_value": rank_value / w_d,
    "instances_tested": 1,
    "conjecture_holds": rank_value <= 1.5 * w_d,
    "counterexample": "" if rank_value <= 1.5 * w_d else "Rank exceeds 1.5 * w^d"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(3, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        support_fraction = len(results) / len(seeds)
        mean = sum(r["metric_value"] for r in results) / len(results)
        std = math.sqrt(sum((r["metric_value"] - mean)**2 for r in results) / len(results))
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Rank exceeds 1.5 * w^d\" first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38260410.py", line 113, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38260410.py", line 95, in run_trial
    tensor_product = grothendieck_tensor_product(poly_f, poly_g)
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38260410.py", line 46, in grothendieck_
    result[i + j][k] ^= poly1[i][j] & poly2[k - j]
                     ^^^^^^^^^^^^^^^^^^^^^^^^^

```

TypeError: unsupported operand type(s) for &: 'int' and 'list'

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed during execution, which prevents us from verifying the conjecture's support conditions. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 13650        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 5928         |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 4987         |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 5699         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15608        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13473        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10488        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 12671        |
| 9 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 8121         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 90626 ms total latency. Provider mix: {'ollama\_remote': 9}

*(full prompt+response transcripts available in research/audit/fffb7ed742253.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/fffb7ed742253.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/fffb7ed742253.tar.gz (if generated)